

Sampling

Temperature, top-p, and why answers vary · video 2:24

This lesson answers one question: why can the same prompt give different answers? At every step the model gives a **probability to every token**, and a **decoding rule** picks one. **Greedy** always takes the top token; **sampling** draws at random by probability. **Temperature** sharpens or flattens the probabilities, and **top-p** cuts the long tail of unlikely tokens.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Real next-token probabilities	▶ 0:08-0:40	3	☐
2. Greedy decoding and sampling	▶ 0:40-1:04	3	☐
3. Temperature	▶ 1:04-1:27	3	☐
4. Top-p: cutting the long tail	▶ 1:27-1:51	3	☐
5. Choosing settings	▶ 1:51-2:22	3	☐

Before you start: How LLMs Work episode 10 (logits, softmax, choosing the next token) and Foundations F06-F07 (probability, sampling and softmax). Code: `code/p03_sampling` (downloads the 0.5B model, about 1 GB, and runs on a CPU).

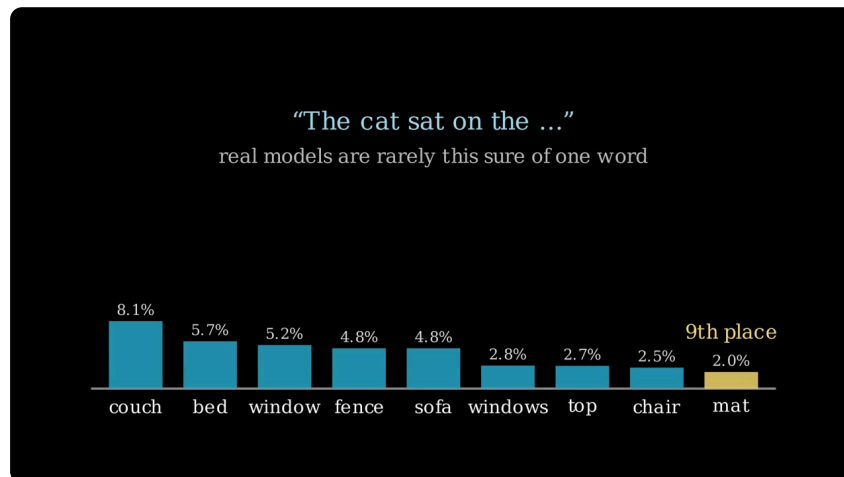
CONCEPT 1 OF 5

Real next-token probabilities

▶ Video 0:08-0:40

Ask a model the same question twice and you can get two different answers. That is not a bug: it is a choice made at every token, called **sampling**.

At each step the model scores every token in its vocabulary (**151,936** for Qwen2.5), and **softmax** turns the scores into probabilities. The real top choices after “*The cat sat on the*” are **couch 8.1%**, bed 5.7%, window 5.2%; **mat** gets only **2.0%**, in ninth place. Real models are rarely as sure as tidy textbook examples.



▶ 0:39 Real probabilities from Qwen2.5-0.5B-Instruct after “The cat sat on the”: couch 8.1%, and “mat” only 2.0%, in 9th place.

KEY IDEA

The model outputs a **distribution**, not a word. Choosing the word is a separate step, and you control it.

Check 1: Real next-token probabilities

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 TRUE OR FALSE

“The model itself decides the next word; the app only displays it.”

True ☐ False ☐

1.2 CALCULATION

After “The cat sat on the”, the top 9 tokens have probabilities 8.1, 5.7, 5.2, 4.8, 4.8, 2.8, 2.7, 2.5 and 2.0 percent. What percentage is left for the other 151,927 tokens?

1.3 SHORT ANSWER

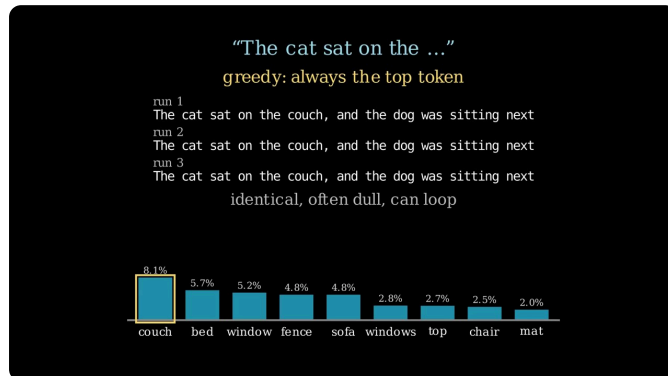
Why does “mat” come only ninth, although it is the classic example?

CONCEPT 2 OF 5

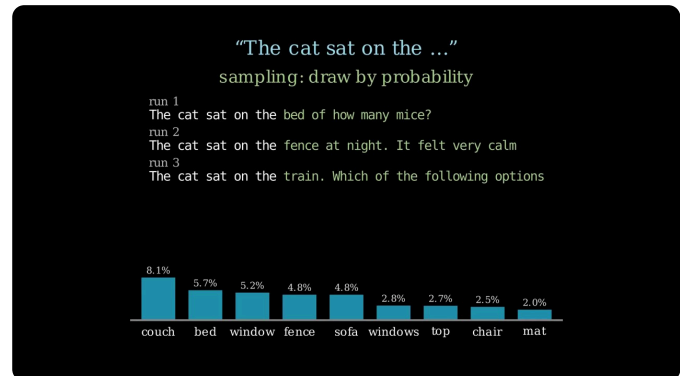
Greedy decoding and sampling

▶ Video 0:40–1:04

The simplest rule is to always pick the top token: **greedy decoding**. Same input, same output, every time: *“The cat sat on the couch, and the dog was sitting next”*. But greedy text tends to be dull, and it can get stuck repeating itself.



▶ 0:51 Greedy: always the top token, so every run is identical.



▶ 1:03 Sampling: draw by probability, so three runs give three different stories (real outputs, seeds 1–3).

Sampling draws a token at random, weighted by its probability: couch 8.1% of the time, bed 5.7%, and so on. Each step's choice changes everything after it, so three runs give three different stories.

KEY IDEA

Greedy = deterministic and safe but flat. Sampling = varied, because every step is a weighted random draw.

Check 2: Greedy decoding and sampling

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 MULTIPLE CHOICE

You run greedy decoding on the same prompt three times. What do you get?

- A. Three different texts
- B. The same text three times
- C. An error
- D. Only the first token

2.2 CALCULATION

With plain sampling (temperature 1), how many times out of 1,000 would you expect the first token to be “couch” (8.1%)?

2.3 SHORT ANSWER

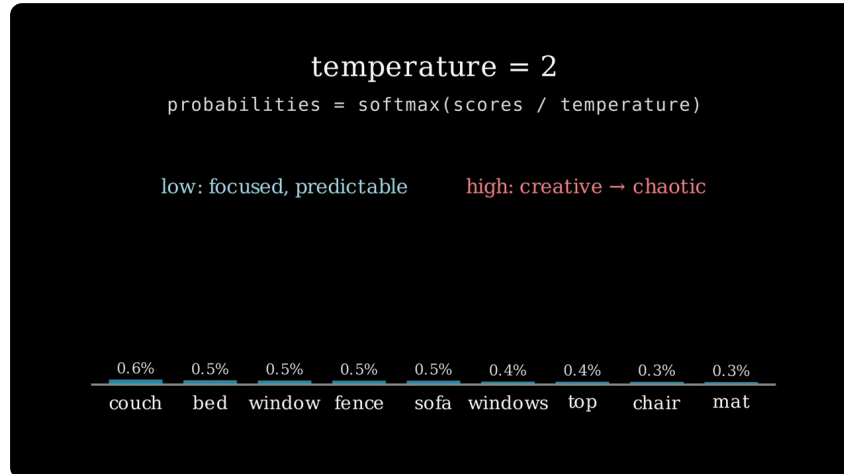
Why does one different early token lead to a completely different story?

CONCEPT 3 OF 5

Temperature

▶ Video 1:04-1:27

Temperature reshapes the odds: divide the scores by the temperature before the softmax. At **0.5** the distribution gets sharper: couch jumps from 8.1% to **29.2%**. At **2** it flattens: couch falls to **0.6%**, and every token is a long shot.



▶ 1:25 At temperature 2 the distribution is almost flat: couch falls to 0.6%.

Low temperature is focused and predictable; high temperature is creative, and then chaotic. Temperature 0 is usually treated as greedy.

KEY IDEA

$\text{softmax}(\text{scores} / T)$: $T < 1$ sharpens, $T > 1$ flattens. The ranking of tokens never changes, only how peaked it is.

Check 3: Temperature

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

Three tokens have scores 2, 1 and 0. At temperature 1 the softmax gives 0.665, 0.245 and 0.090. What is the top token's probability at temperature 0.5? (Hint: the scores become 4, 2, 0.)

3.2 MULTIPLE CHOICE

Same scores, temperature 2. Which is closest to the top token's probability?

- A. 0.87
- B. 0.67
- C. 0.51
- D. 0.33

3.3 TRUE OR FALSE

"Raising the temperature can make a token that was ranked 5th become ranked 1st."

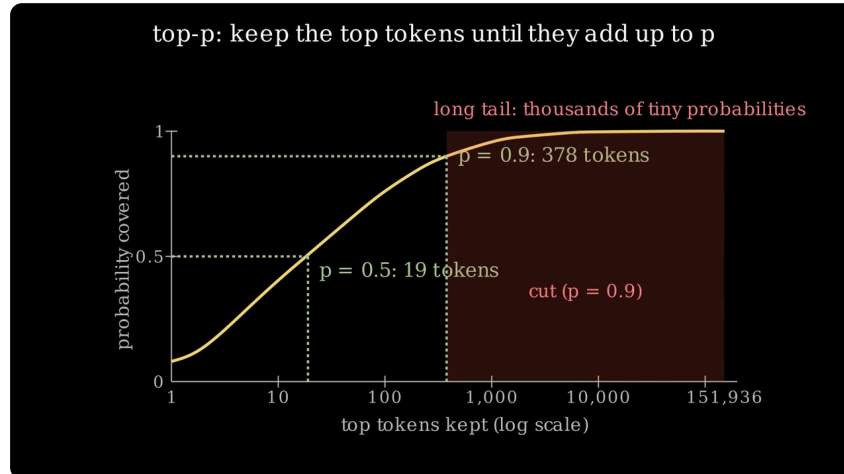
True ☐ False ☐

CONCEPT 4 OF 5

Top-p: cutting the long tail

▶ Video 1:27-1:51

The **long tail**: thousands of unlikely tokens, each tiny, but together they add up, and some of them are nonsense. **Top-p** (nucleus) sampling keeps only the smallest set of top tokens whose probabilities add up to **p**, renormalises them, and samples from those.



▶ 1:49 Cumulative probability of the top tokens (log scale): top-p 0.5 keeps 19 tokens, top-p 0.9 keeps 378 of 151,936.

For our prompt, top-p 0.9 keeps **378** tokens and top-p 0.5 keeps just **19**. Everything else is cut. How many tokens survive depends on how sure the model is: a confident model may keep only one or two.

KEY IDEA

Top-p adapts to the model's certainty: it keeps few tokens when the model is sure, and many when it is not.

Check 4: Top-p: cutting the long tail

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

Four tokens have probabilities 0.5, 0.3, 0.15 and 0.05. With top-p = 0.8 (keep a token while the mass before it is below p), which tokens are kept, and what are their probabilities after renormalising?

4.2 TRUE OR FALSE

"Top-p 0.9 always keeps the same number of tokens."

True ☐ False ☐

4.3 PUT IN ORDER

Put the top-p steps in order: *sample · sort by probability · renormalise · add up until p.*

CONCEPT 5 OF 5

Choosing settings

▶ Video 1:51-2:22

In code: divide the logits by the temperature and take the softmax; sort the probabilities, add them up, keep tokens until you reach p; then draw one at random.

```
1 probs = softmax(logits / temperature)
2
3 order = argsort(probs, descending=True)
4 mass = cumsum(probs[order])
5 keep = order[mass - probs[order] < top_p] # nucleus
6
7 token = random.choices(keep, weights=probs[keep])[0]
```

Full version: code/p03_sampling

facts · code · extraction
low temperature (0 - 0.3)
close to greedy

brainstorming · stories
higher temperature (0.8 - 1.2)
plus top-p ≈ 0.9

▶ 2:00 Temperature, then sort, cumulative sum, cut at p, and draw.

▶ 2:16 Typical starting points: low temperature for precise tasks, higher with top-p for creative ones.

For **facts, code and extracting data**: low temperature, close to greedy. For **brainstorming and stories**: a higher temperature, with top-p to trim the nonsense. For **repeatable tests**: fix the random seed.

KEY IDEA

Pick decoding settings per task: precision wants low randomness, creativity wants more, and testing wants a fixed seed.

Check 5: Choosing settings

Answer all 3 questions before looking at the answers. Pass mark: all correct.

5.1 MULTIPLE CHOICE

You extract invoice totals from emails into a spreadsheet. Which setting?

- A. Temperature 1.5, top-p 1.0
- B. Temperature 0 (greedy)
- C. Temperature 1.2, top-p 0.9
- D. A different random seed for each email

5.2 SHORT ANSWER

Why fix the random seed when testing a prompt that uses sampling?

5.3 HANDS-ON CODE

Try it yourself. In `code/p03_sampling/sampling.py`, change `PROMPT` to "The capital of France is" and run it. (a) What is the top token and its probability at $T = 1$? (b) How many tokens does top-p 0.9 keep now? (c) Explain the difference from the cat prompt.

```
PROMPT = "The capital of France is"
# then run: python sampling.py
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Real next-token probabilities · rewatch ▶ 0:08-0:40

1.1 False. The model outputs probabilities; a decoding rule (greedy, sampling, ...) set by the app picks the token.

1.2 About 61.4%. $100 - 38.6 = 61.4$: most of the probability is spread over the long tail.

1.3 Many continuations are plausible (couch, bed, window, ...); the model spreads probability over all of them. Without more context, “mat” is just one of many reasonable endings.

2. Greedy decoding and sampling · rewatch ▶ 0:40-1:04

2.1 B. Greedy always picks the top token, so nothing is random.

2.2 About 81. $1,000 \times 0.081 = 81$.

2.3 Each new token is appended and becomes part of the input for every later step. That is the generation loop of How LLMs Work episode 1: predict, pick, append, repeat.

3. Temperature · rewatch ▶ 1:04-1:27

3.1 0.867. $e^4 / (e^4 + e^2 + e^0) = 54.6 / 63.0 \approx 0.867$: lower temperature sharpens.

3.2 C. Scores 1, 0.5, 0 give 0.506: flatter, moving toward 1/3 each.

3.3 False. Dividing all scores by the same positive number keeps their order.

4. Top-p: cutting the long tail · rewatch ▶ 1:27-1:51

4.1 The first two: 0.625 and 0.375. Mass before the 3rd token is 0.8, not below 0.8, so it is cut. $0.5 / 0.8 = 0.625$; $0.3 / 0.8 = 0.375$.

4.2 False. It keeps as many as needed to reach 90% of the probability, which depends on the distribution at that step.

4.3 sort by probability → add up until p → renormalise → sample. This is what the episode's code does.

5. Choosing settings · rewatch ▶ 1:51-2:22

5.1 B. You want the single most likely, consistent answer, not variety.

5.2 So the same input gives the same output, and changes in results come from your prompt changes, not from chance. With a fixed seed the random draws repeat exactly.

5.3 (a) “ Paris”, about 0.30 (next come quiz-style blanks such as “ _____”, 0.12). (b) 25 tokens. (c) The model is much surer about a fact than about where a cat sat, so probability is concentrated on a few tokens and top-p 0.9 needs 25 instead of 378. Top-p adapts to certainty. The blanks show the model also considers that the text might be a quiz, like a base model would (How LLMs Work, episode 14).